

ORACLE HR SCHEMA

SQL Aggregation Functions — Practice Guide

Schema	HR
Tables	EMPLOYEES · DEPARTMENTS · JOBS · JOB_HISTORY · LOCATIONS · COUNTRIES · REGIONS
Tool	Oracle SQL Developer
Topics	COUNT · SUM · AVG · MAX · MIN · GROUP BY · HAVING

SECTION 1 — COUNT

COUNT() counts every row. COUNT(column) counts only rows where that column is NOT NULL.*

1a Count the total number of employees

```
SELECT COUNT(*) AS total_employees  
FROM HR.EMPLOYEES;
```

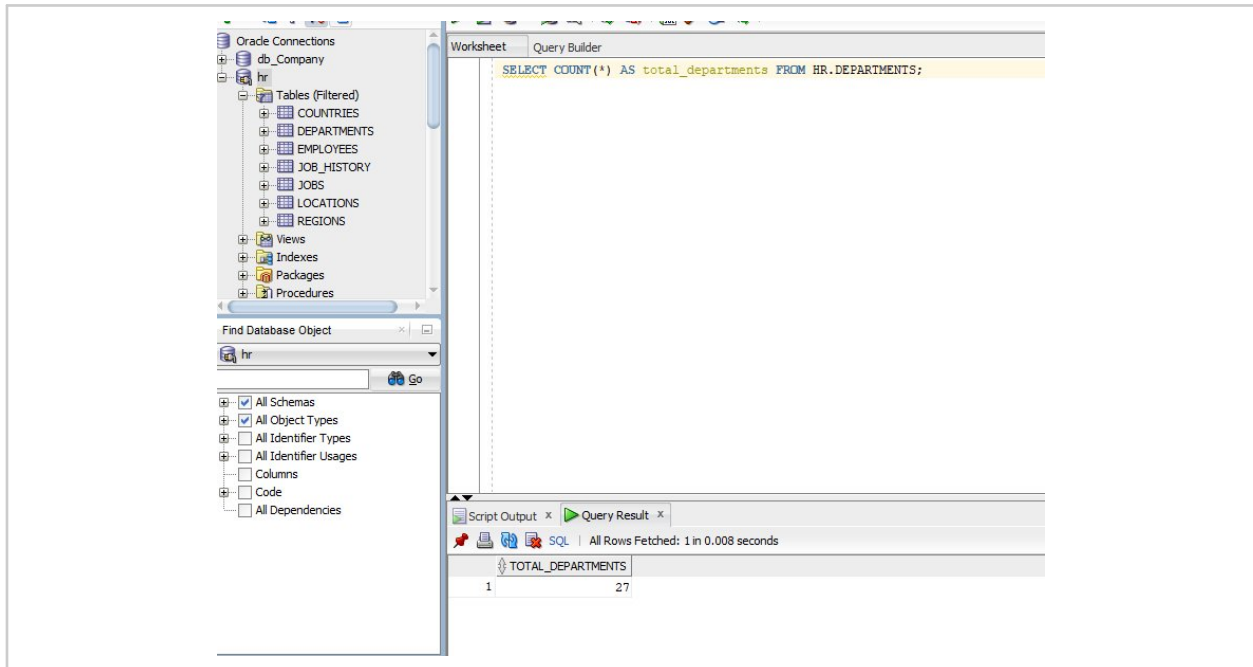
Note: Expected result: 107

The screenshot displays the Oracle SQL Developer interface. On the left, the 'Connections' pane shows the 'hr' schema selected under 'db_Company'. The 'Tables (Filtered)' list includes COUNTRIES, DEPARTMENTS, EMPLOYEES, JOB_HISTORY, JOBS, LOCATIONS, and REGIONS. The 'Query Builder' pane shows the query: `SELECT COUNT(*) AS total_employees FROM HR.EMPLOYEES;`. The 'Query Result' pane at the bottom shows the execution results: 'TOTAL_EMPLOYEES' with a value of 107. The status bar indicates 'All Rows Fetched: 1 in 0.009 seconds'.

1b Count the total number of departments

```
SELECT COUNT(*) AS total_departments  
FROM HR.DEPARTMENTS;
```

Note: Expected result: 27

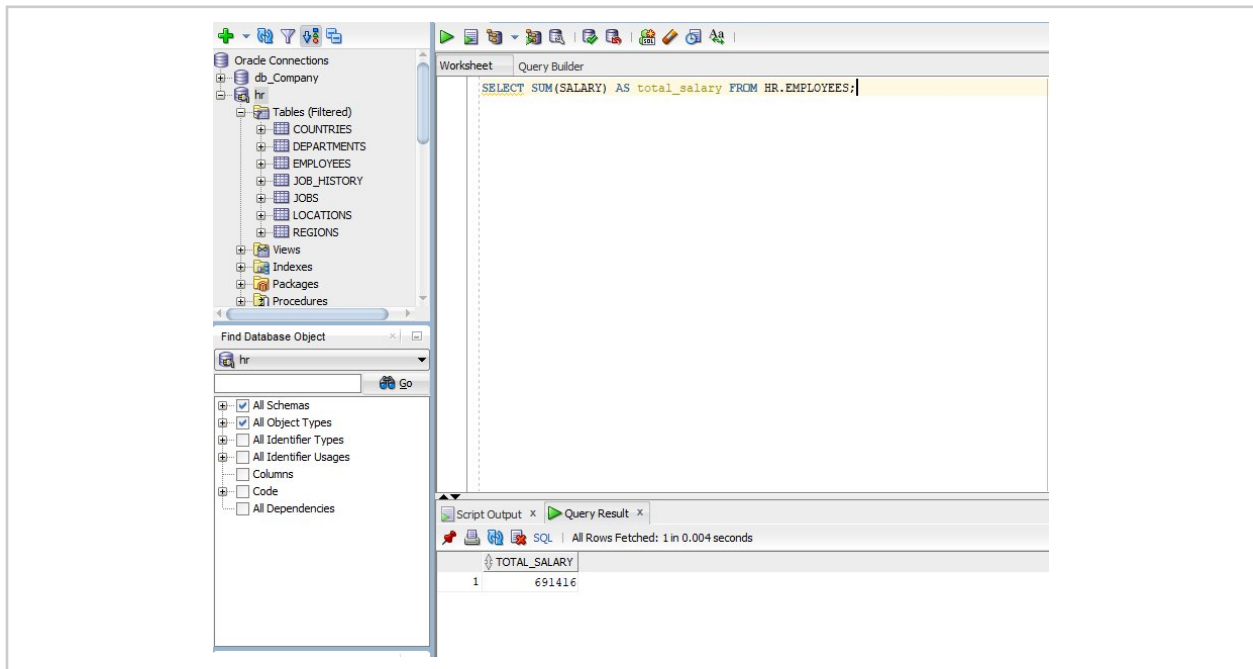


SECTION 2 — SUM

SUM adds up all values in a numeric column. NULL values are ignored automatically.

2a Calculate the total salary paid across all employees

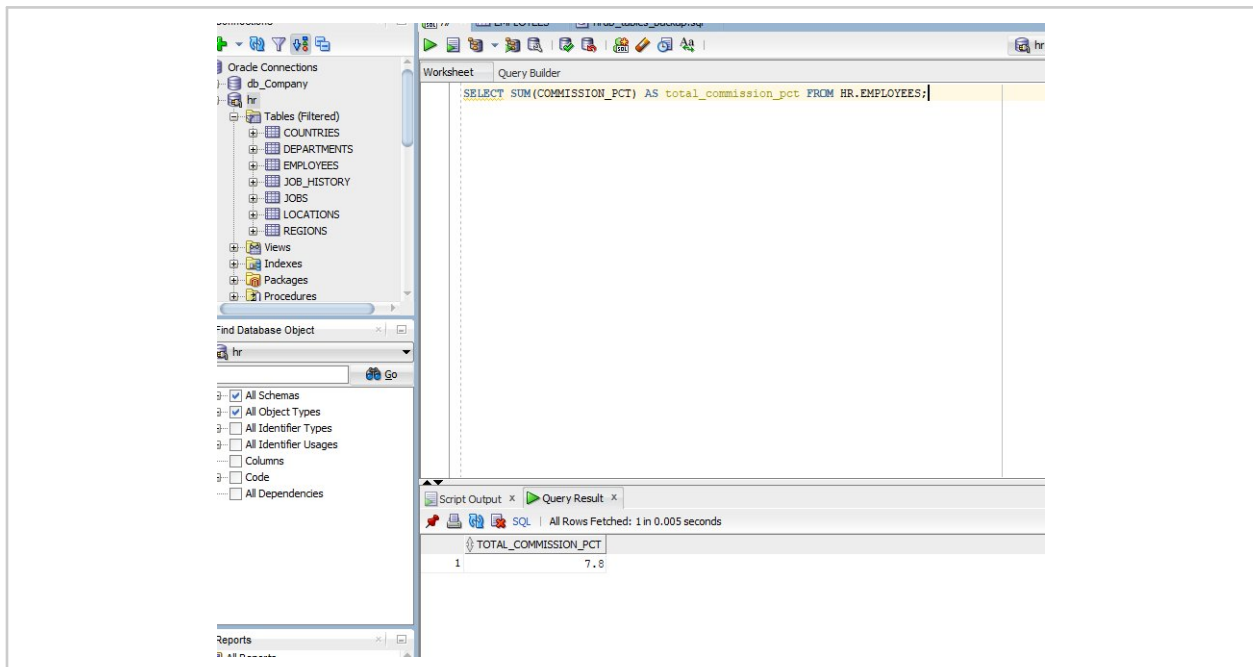
```
SELECT SUM(SALARY) AS total_salary  
FROM HR.EMPLOYEES;
```



2b Calculate the total commission percentage across all employees

```
SELECT SUM(COMMISSION_PCT) AS total_commission_pct
FROM HR.EMPLOYEES;
```

Note: COMMISSION_PCT is NULL for non-sales employees — SUM skips those NULLs automatically.

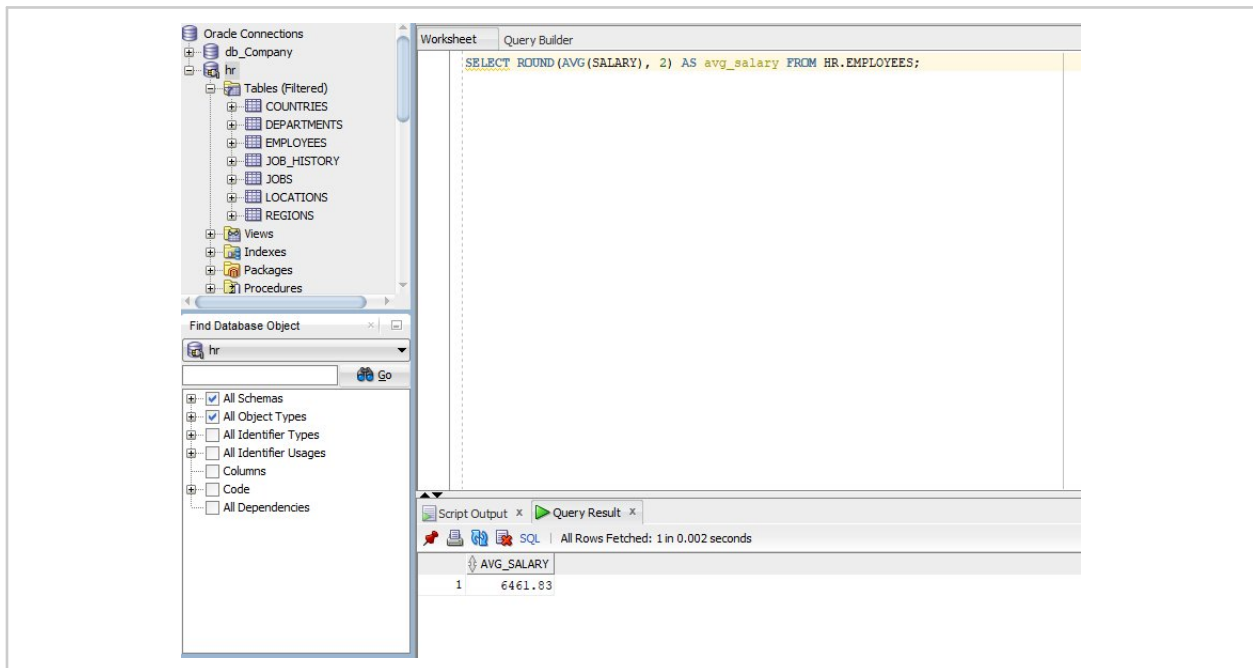


SECTION 3 — AVG

AVG calculates the mean value of a numeric column. ROUND(value, 2) keeps 2 decimal places. NULL values are ignored automatically.

3a Find the average salary of all employees

```
SELECT ROUND(AVG(SALARY), 2) AS avg_salary  
FROM HR.EMPLOYEES;
```

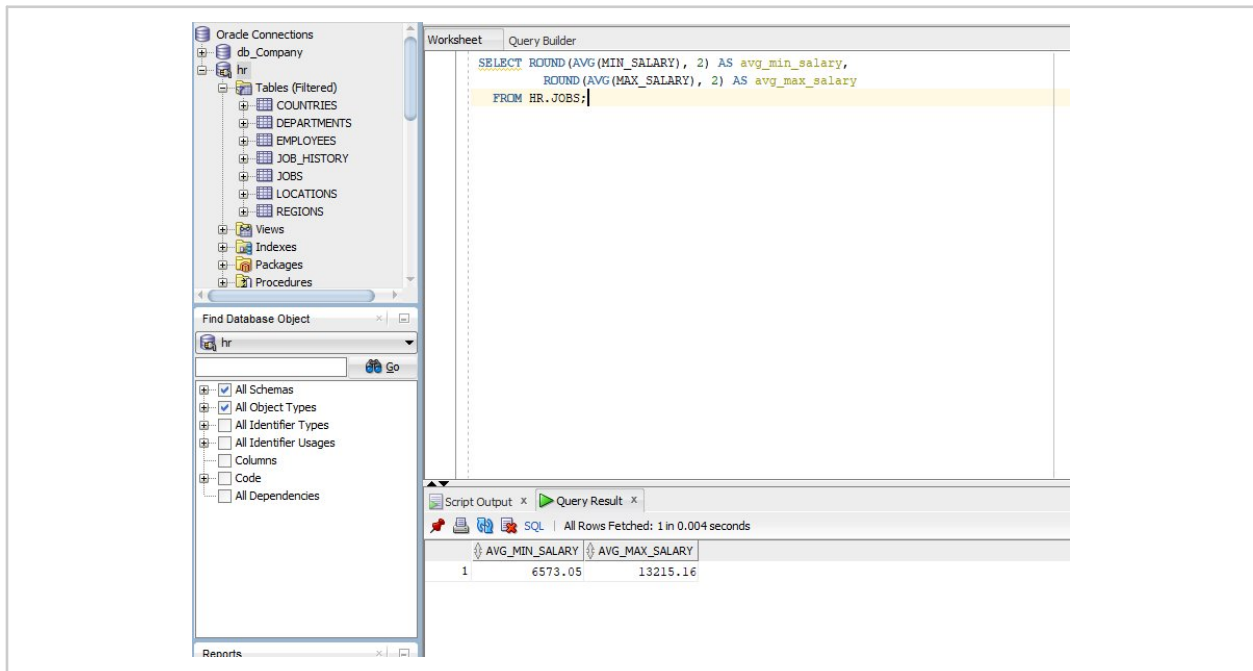


The screenshot displays the Oracle SQL Developer interface. On the left, the 'Oracle Connections' tree shows the 'hr' schema under 'db_Company'. The 'Find Database Object' pane is open, showing a search for 'hr'. The 'Worksheet' pane on the right contains the SQL query: `SELECT ROUND(AVG(SALARY), 2) AS avg_salary FROM HR.EMPLOYEES;`. Below the query, the 'Query Result' pane shows the output: a single row with the column header 'AVG_SALARY' and the value '6461.83'. The status bar at the bottom indicates 'All Rows Fetched: 1 in 0.002 seconds'.

AVG_SALARY
6461.83

3b Find average minimum and maximum salary from the JOBS table

```
SELECT ROUND(AVG(MIN_SALARY), 2) AS avg_min_salary,  
       ROUND(AVG(MAX_SALARY), 2) AS avg_max_salary  
FROM HR.JOBS;
```



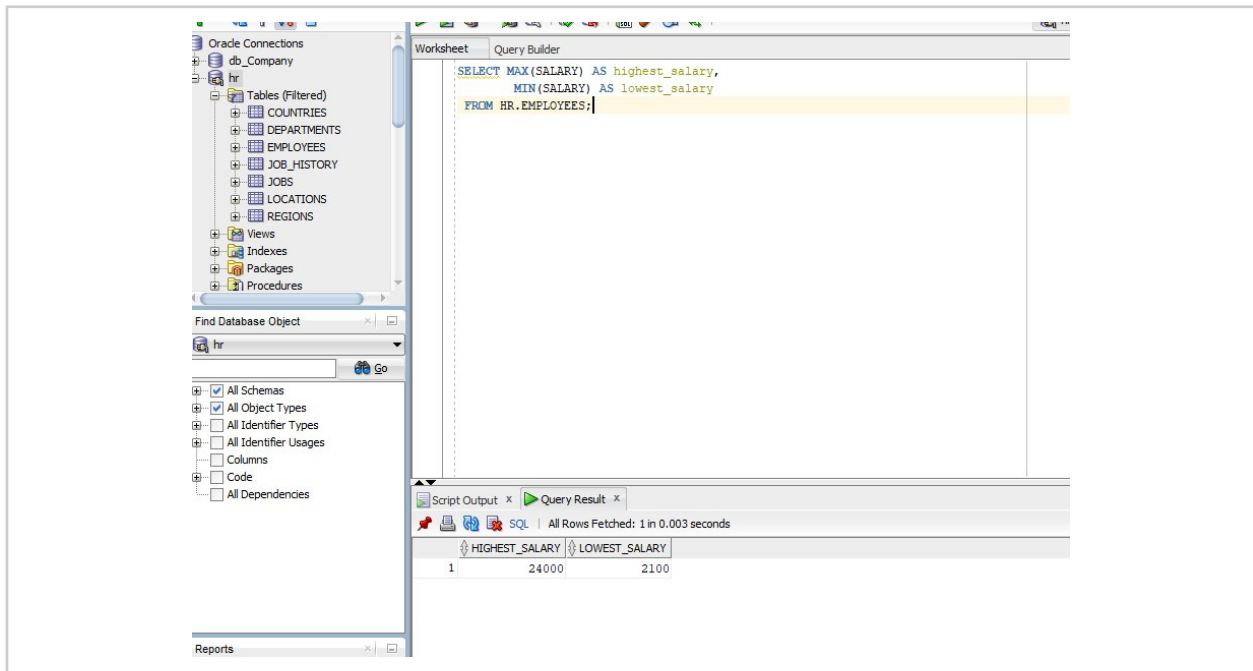
SECTION 4 — MAX and MIN

MAX returns the largest value in a column. MIN returns the smallest value in a column.

4a Retrieve the highest and lowest salary from EMPLOYEES

```
SELECT MAX(SALARY) AS highest_salary,  
       MIN(SALARY) AS lowest_salary  
FROM HR.EMPLOYEES;
```

Note: Expected: highest = 24000 (Steven King, President), lowest = 2100



SECTION 5 — GROUP BY

GROUP BY groups rows that share the same value so you can apply aggregate functions per group.

5a Count employees per department

```
SELECT DEPARTMENT_ID,  
       COUNT(*) AS employee_count  
FROM HR.EMPLOYEES  
GROUP BY DEPARTMENT_ID  
ORDER BY DEPARTMENT_ID;
```

Note: One employee (Kimberely Grant, emp_id 178) has no department — she appears as NULL.

Query Builder

```
SELECT DEPARTMENT_ID,
COUNT(*) AS employee_count
FROM HR.EMPLOYEES
GROUP BY DEPARTMENT_ID
ORDER BY DEPARTMENT_ID;
```

Script Output: Query Result

DEPARTMENT_ID	EMPLOYEE_COUNT
10	1
20	2
30	6
40	1
50	45
60	5
70	1
80	34
90	3
100	6
110	2
(null)	1

5b Show average min/max salary per job title

```
SELECT JOB_TITLE,
       AVG(MIN_SALARY) AS avg_min_salary,
       AVG(MAX_SALARY) AS avg_max_salary
FROM HR.JOBS
GROUP BY JOB_TITLE
ORDER BY JOB_TITLE;
```

Query Builder

```
SELECT JOB_TITLE,
       AVG(MIN_SALARY) AS avg_min_salary,
       AVG(MAX_SALARY) AS avg_max_salary
FROM HR.JOBS
GROUP BY JOB_TITLE
ORDER BY JOB_TITLE;
```

Script Output: Query Result

JOB_TITLE	AVG_MIN_SALARY	AVG_MAX_SALARY
1 Accountant	4200	9000
2 Accounting Manager	8200	16000
3 Administration Assistant	3000	6000
4 Administration Vice President	15000	30000
5 Finance Manager	8200	16000
6 Human Resources Representative	4000	9000
7 Marketing Manager	9000	15000
8 Marketing Representative	4000	9000
9 President	20080	40000
10 Programmer	4000	10000
11 Public Accountant	4200	9000
12 Public Relations Representative	4500	10500
13 Purchasing Clerk	2500	5500
14 Purchasing Manager	8000	15000
15 Sales Manager	10000	20080
16 Sales Representative	6000	12008
17 Shipping Clerk	2500	5500
18 Stock Clerk	2008	5500
19 Stock Manager	5500	8500

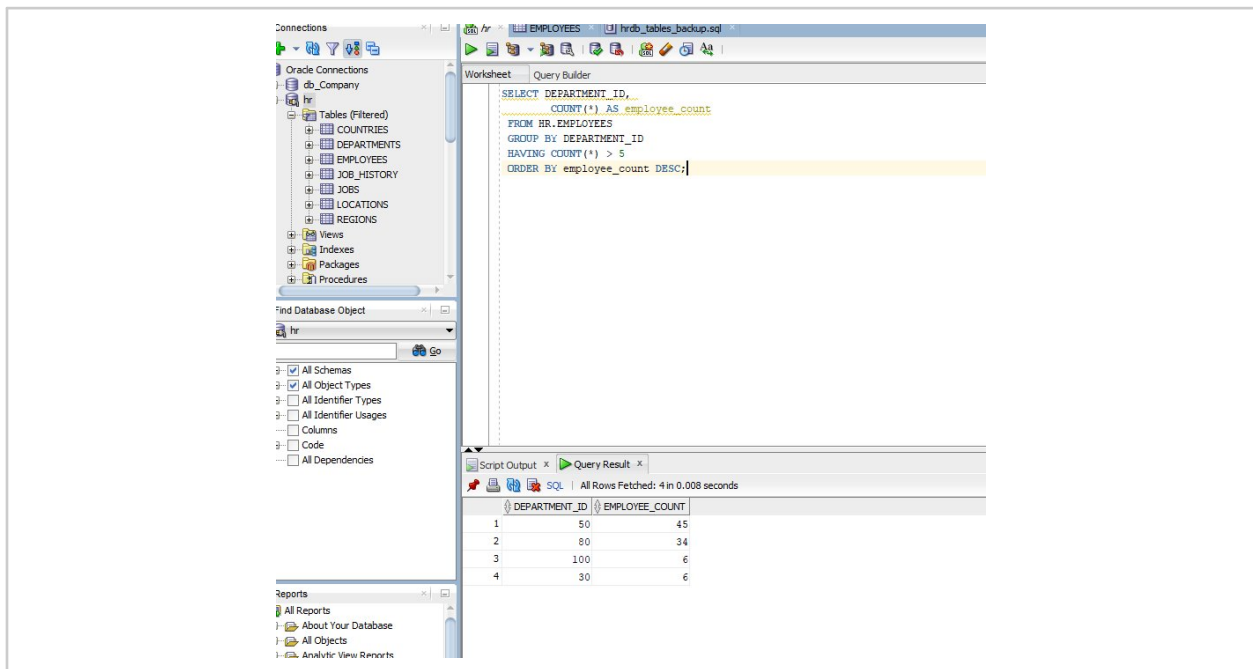
SECTION 6 — HAVING

HAVING filters groups AFTER aggregation. WHERE filters rows BEFORE grouping.

6a Show only departments with more than 5 employees

```
SELECT DEPARTMENT_ID,  
       COUNT(*) AS employee_count  
FROM HR.EMPLOYEES  
GROUP BY DEPARTMENT_ID  
HAVING COUNT(*) > 5  
ORDER BY employee_count DESC;
```

Note: Departments with 5 or fewer employees are excluded from the result.



The screenshot displays the Oracle SQL Developer interface. The 'Query Builder' window shows the following SQL query:

```
SELECT DEPARTMENT_ID,  
       COUNT(*) AS employee_count  
FROM HR.EMPLOYEES  
GROUP BY DEPARTMENT_ID  
HAVING COUNT(*) > 5  
ORDER BY employee_count DESC;
```

The 'Query Result' window shows the output of the query:

DEPARTMENT_ID	EMPLOYEE_COUNT
50	45
80	34
100	6
30	6

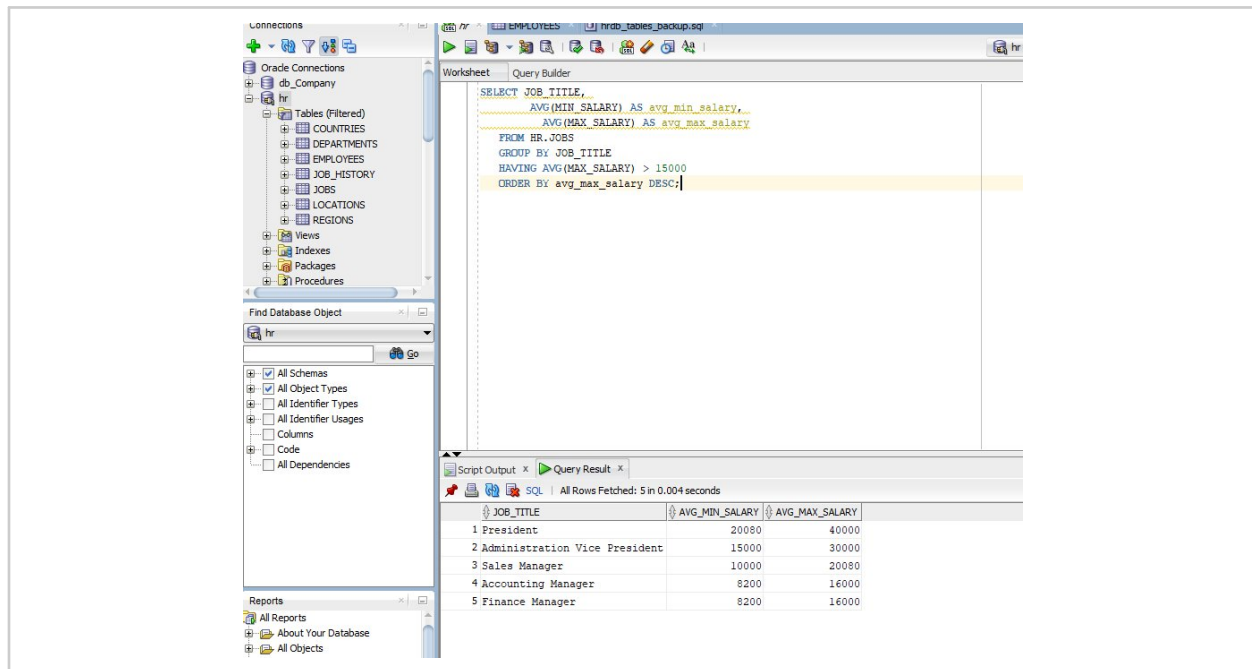
6b Show job titles where average MAX_SALARY exceeds 15,000

```
SELECT JOB_TITLE,  
       AVG(MIN_SALARY) AS avg_min_salary,  
       AVG(MAX_SALARY) AS avg_max_salary  
FROM HR.JOBS  
GROUP BY JOB_TITLE
```



```
HAVING AVG(MAX_SALARY) > 15000  
ORDER BY avg_max_salary DESC;
```

Note: Filters out lower-paying job categories.



The screenshot displays the Oracle SQL Developer interface. On the left, the 'Connections' pane shows the 'hr' schema selected. The 'Query Builder' pane on the right contains the following SQL query:

```
SELECT JOB_TITLE,  
       AVG(MIN_SALARY) AS avg_min_salary,  
       AVG(MAX_SALARY) AS avg_max_salary  
FROM HR.JOBS  
GROUP BY JOB_TITLE  
HAVING AVG(MAX_SALARY) > 15000  
ORDER BY avg_max_salary DESC;
```

Below the query, the 'Script Output' pane shows the query results. The results are displayed in a table with three columns: JOB_TITLE, AVG_MIN_SALARY, and AVG_MAX_SALARY. The data is as follows:

JOB_TITLE	AVG_MIN_SALARY	AVG_MAX_SALARY
1 President	20080	40000
2 Administration Vice President	15000	30000
3 Sales Manager	10000	20080
4 Accounting Manager	8200	16000
5 Finance Manager	8200	16000